

PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE

TOPIC:

QR CODE-BASED IN-TABLE FOOD ORDERING SYSTEM REPORT

REPORT: LAB 6 - INTRODUCING THE API GATEWAY PATTERN

Code Course : CSE703110-1-1-25

Class : N01

Instructor : THS. Vũ Quang Dũng

Group : 12

Members : 1. Nguyễn Đình Trung - 23010701
2. Vũ Anh Nam - 23010886
3. Đặng Minh Hoàng – 23010445
4. Nguyễn Phan Quang Minh - 23010530

CONTENTS

1. Abstract.....	3
2. Objectives.....	3
3. System Overview and Architectural Context	3
3.1 Background	3
4. API Gateway Pattern	4
5. Architecture Design.....	6
5.1. System Architecture Diagram	6
5.2. Routing Strategy	6
6. Technology Stack.....	6
7. API Gateway Implementation 7.1 Gateway Configuration	7
7.2 Security Handling	7
7.3 Request Forwarding.....	7
7.4 Fault Isolation.....	7
8. Testing and Validation	8
8.1 Unauthorized Access Test.....	8
8.2 Authorized Access Test.....	8
8.3 Routing Validation Test	8
8.4 Service Isolation Test.....	8
9. Discussion.....	8
10. Conclusion.....	9

1. Abstract

Lab 6 introduces the **API Gateway Pattern** into the QR Code-Based In-Table Food Ordering System (FOODASH). As the platform evolves to handle high-concurrency tableside transactions, managing cross-cutting concerns—such as request routing, authentication, and rate limiting—at the application module level increases architectural coupling. By establishing an independent API Gateway layer, the system provides a single unified entry point for all client devices (Customers, Staff, and Administrators), centralizes edge security policies, and decouples reverse proxy routing mechanics from the core backend operational logic.

2. Objectives

- Design and incorporate an API Gateway layer into the existing modular monolithic architecture of FOODASH.
- Configure static and dynamic reverse proxy routing strategies to separate customer tableside ordering traffic from administrative management services.
- Offload cross-cutting concerns including central JWT authentication token verification and cross-origin resource sharing (CORS) configurations.
- Validate system isolation and architectural stability under simulated network endpoints.

3. System Overview and Architectural Context

3.1 Background

The current MVP layout of FOODASH utilizes a React frontend single-page application (SPA) communicating directly with a Node.js/Express backend API server. While this structure functions adequately under normal load, exposing core backend interfaces directly to public client browsers increases vulnerability risks and complicates path routing rules. Introducing an API Gateway optimizes request dispatching and acts as a protective infrastructure shield over the relational PostgreSQL database and in-memory Redis cache wrappers.

To address these challenges, an API Gateway is introduced as a façade layer that mediates all external access to backend services.

3.2 Architectural Approach

4. API Gateway Pattern

The API Gateway serves as an architectural reverse proxy layer positioned between external client device applications and private downstream backend service modules. Instead of clients calling internal system nodes directly, all inbound HTTP requests hit the gateway, which inspects the request URI and distributes the payload to the appropriate internal destination. This centralizes vital system governance capabilities, preventing duplicate security or routing implementations inside individual backend codebases.

In a tableside ordering context, hundreds of customers scan QR codes simultaneously, generating heavy read traffic on the menu catalog. The API Gateway optimizes this flow by executing non-blocking connection handling. It intercepts malicious parameters, drops invalid paths, manages SSL termination, and routes requests to cached modules before the traffic hits the primary database cluster. This architecture isolates core business logic layers and prevents system-wide cascading crashes.

5. Architecture Design

5.1. System Architecture Diagram

The API Gateway serves as the centralized interface shell for FOODASH. All user channels interact exclusively with the Nginx edge server, which acts as the perimeter barrier before routing traffic to the internal Node.js modules.

5.2. Detailed Routing Strategy

The gateway employs a path-based segmentation strategy to route traffic to the appropriate application context:

- **Public Customer Routes (/api/v1/menu/*):** High-throughput, non-blocking paths optimized for quick item presentation. These endpoints are linked to the Redis layer to keep response times under 2 seconds.
- **Transactional Ordering Routes (/api/v1/orders/*):** Paths that process active shopping carts, table status changes, and staff notification alerts.
- **Administrative Control Routes (/api/v1/admin/*):** Restricted dashboard endpoints that require strict token inspection before access is granted.

6. Technology Stack

To meet the high-concurrency and performance demands of the restaurant platform, the following production-grade tools were selected:

- **Nginx Server:** Chosen as the primary API Gateway engine due to its asynchronous, event-driven architecture, low memory footprint, and high performance as a reverse proxy capable of handling over 10,000 concurrent requests.
- **Node.js & Express.js:** Used to build the core backend application layer, providing rapid JSON processing capabilities.
- **PostgreSQL & Redis:** Selected to handle persistent relational storage and fast in-memory query acceleration, respectively.

7. Implementation Details

The Nginx reverse proxy is configured to expose a unified interface on port 80, routing inbound traffic to the underlying application servers running on internal ports. Below is the comprehensive production-grade Nginx configuration file tailored for the FOODASH architecture:

8. Discussion

The introduction of the API Gateway Pattern significantly improves the architectural quality of the FOODASH system. By offloading cross-cutting perimeter concerns—such as dynamic CORS pre-flight matching, static response header injections, and routing rules—to Nginx, the core backend services can focus entirely on executing business logic.

Furthermore, this architecture ensures high production readiness. If a massive user surge during peak restaurant hours requires scaling the platform, the client-side browser applications will remain completely unaffected. If the *Order & Assistance Real-Time Workflow Module* needs to be extracted into a standalone service on a separate server due to heavy traffic, the migration can be managed through minor upstream rule updates

within the Nginx configuration file. The frontend clients can continue calling the same base URL string without requiring any codebase refactoring.

9. Conclusion

Lab 6 successfully implements the **API Gateway Pattern** within the QR Code-Based In-Table Food Ordering System (FOODASH). The Nginx-engineered perimeter container provides a centralized entry point that abstracts system complexity, balances traffic routing, and enforces clean boundary controls. This implementation reinforces key software engineering principles—including the separation of concerns, loose structural coupling, and secure edge filtering—ensuring that the FOODASH platform is performant, maintainable, and well-prepared for future distributed scaling.